Experiment 1: Installation of Reinforcement Learning Development Environment

Program:

```
import tensorflow as tf

import keras

import gymnasium as gym

import numpy as np

import matplotlib.pyplot as plt

print("TensorFlow Version:", tf.__version__)

print("Keras Version:", keras.__version__)

print("NumPy Version:", np.__version__)

env = gym.make("CartPole-v1")

state, info = env.reset()

print("Initial State:")

print(state)

action = env.action_space.sample()


next_state, reward, terminated, truncated, info = env.step(action)
```

```
print("Action:", action)

print("Reward:", reward)

print("Next State:")

print(next_state)

env.close()

print("Reinforcement Learning Environment Verified Successfully!")
```

Output:

```
>
========== RESTART: /Users/vinnu/Documents/ep-1.py ========
TensorFlow Version: 2.21.0
Keras Version: 3.15.0
NumPy Version: 2.3.2
Initial State:
[ 0.02957765  0.0466412  -0.02422945  0.00348538]
Action: 0
Reward: 1.0
Next State:
[ 0.03051048 -0.14812504 -0.02415974  0.28842622]
Reinforcement Learning Environment Verified Successfully!
>
```

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Program:

```
import gymnasium as gym
environments = ["FrozenLake-v1", "CartPole-v1", "MountainCar-v0"]
for env_name in environments:
    print("\n==============================")
    print("Environment:", env_name)
    print("==============================")
    env = gym.make(env_name)
    state, info = env.reset()
    print("Initial State:", state)
    done = False
    step = 1
    while not done and step <= 3:
        action = env.action_space.sample()
        next_state, reward, terminated, truncated, info = env.step(action)
        print("\nStep:", step)
        print("Action:", action)
        print("Next State:", next_state)
        print("Reward:", reward)
```

```
    done = terminated or truncated
    step += 1
print("\nEpisode Finished")
env.close()
```

Output:

```
==============================
Environment: FrozenLake-v1
==============================
Initial State: 0

Step: 1
Action: 0
Next State: 0
Reward: 0

Step: 2
Action: 2
Next State: 1
Reward: 0

Step: 3
Action: 2
Next State: 5
Reward: 0

Episode Finished

==============================
Environment: CartPole-v1
==============================
Initial State: [ 0.02018492  0.01707623 -0.00466908  0.04360532]

Step: 1
Action: 1
Next State: [ 0.02052645  0.21226482 -0.00379697 -0.25054708]
Reward: 1.0

Step: 2
Action: 1
Next State: [ 0.02477174  0.40744078 -0.00880792 -0.54442525]
Reward: 1.0

Step: 3
Action: 1
Next State: [ 0.03292056  0.6026854  -0.01969642 -0.8398703 ]
Reward: 1.0

Episode Finished

==============================
Environment: MountainCar-v0
```

```
Episode Finished

==============================
Environment: MountainCar-v0
==============================
Initial State: [-0.54530895  0.        ]

Step: 1
Action: 0
Next State: [-0.5461462  -0.00083729]
Reward: -1.0

Step: 2
Action: 0
Next State: [-0.54781455 -0.00166831]
Reward: -1.0

Step: 3
Action: 1
Next State: [-0.5493014  -0.00148685]
Reward: -1.0

Episode Finished
> 
```

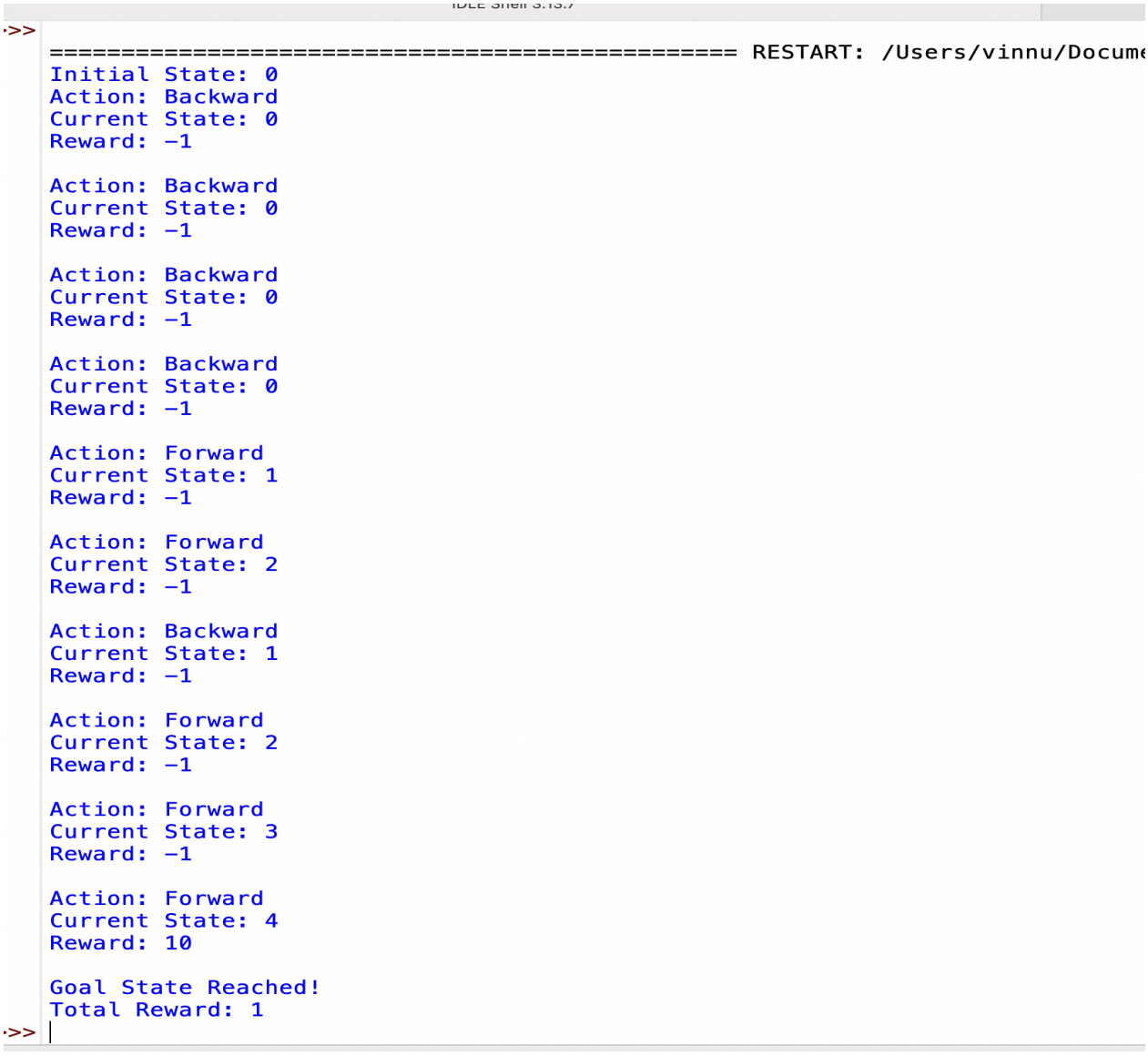
Experiment 3: Implementation of the Reinforcement Learning Framework

Program:

```
import random
states = [0, 1, 2, 3, 4]
goal = 4
state = 0
total_reward = 0
print("Initial State:", state)
while state != goal:
  action = random.choice(["Forward", "Backward"])
   if action == "Forward":
      if state < goal:
         state += 1
   else:
      if state > 0:
         state -= 1
   if state == goal:
      reward = 10
   else:
      reward = -1
   total_reward += reward
```

```
    print("Action:", action)
    print("Current State:", state)
    print("Reward:", reward)
    print()
print("Goal State Reached!")
print("Total Reward:", total_reward)
```

output:



Experiment 4: Simulation of a Markov Decision Process (MDP)

Program:

```
print("Markov Decision Process")
state = "A"
reward = 0
while state != "D":
    if state == "A":
        print("Current State :", state)
        state = "B"
        reward = reward + 5
        print("Next State :", state)
        print("Reward :", 5)
    elif state == "B":
        print("Current State :", state)
        state = "C"
        reward = reward + 3
        print("Next State :", state)
        print("Reward :", 3)
    elif state == "C":
        print("Current State :", state)
        state = "D"
        reward = reward + 10
```

```
        print("Next State :", state)
        print("Reward :", 10)
print("Goal Reached")
print("Total Reward =", reward)
```

output:

```
Total Reward: 1
>
==========================================
Markov Decision Process
Current State : A
Next State : B
Reward : 5
Current State : B
Next State : C
Reward : 3
Current State : C
Next State : D
Reward : 10
Goal Reached
Total Reward = 18
>
```

Experiment 5: Bellman Equation Demonstration

Program:

```
states = ["S1", "S2", "S3"]
reward = {
    "S1": 5,
    "S2": 8,
    "S3": 10
}
gamma = 0.9
value = {
    "S1": 0,
    "S2": 0,
    "S3": 0
}
print("Initial State Values")
for s in states:
    print(s, "=", value[s])
print("\nBellman Expectation Values")
for i in range(5):
    new_value = {}
    new_value["S1"] = reward["S1"] + gamma * value["S2"]
    new_value["S2"] = reward["S2"] + gamma * value["S3"]
```

```
    new_value["S3"] = reward["S3"]
    value = new_value
    print("\nIteration", i + 1)
    for s in states:
        print(s, "=", round(value[s], 2)
print("\nBellman Optimality Values")
q1 = reward["S1"] + gamma * max(value["S2"], value["S3"])
q2 = reward["S2"] + gamma * value["S3"]
q3 = reward["S3"]
print("Q(S1) =", round(q1, 2))
print("Q(S2) =", round(q2, 2))
print("Q(S3) =", round(q3, 2))
```

Output:

```
>>
========== RESTART: /Users/vinnu/Documer
Initial State Values
S1 = 0
S2 = 0
S3 = 0

Bellman Expectation Values

Iteration 1
S1 = 5.0
S2 = 8.0
S3 = 10

Iteration 2
S1 = 12.2
S2 = 17.0
S3 = 10

Iteration 3
S1 = 20.3
S2 = 17.0
S3 = 10

Iteration 4
S1 = 20.3
S2 = 17.0
S3 = 10

Iteration 5
S1 = 20.3
S2 = 17.0
S3 = 10

Bellman Optimality Values
Q(S1) = 20.3
Q(S2) = 17.0
Q(S3) = 10
```

Experiment 6: Exploration vs. Exploitation Strategies

Program:

```
import random
actions = ["A", "B", "C"]
reward = {
    "A": 5,
    "B": 8,
    "C": 10
}
epsilon = 0.3
print("Greedy Strategy")
best = max(reward, key=reward.get)
print("Selected Action:", best)
print("Reward:", reward[best])
print("\nRandom Strategy")
r = random.choice(actions)
print("Selected Action:", r)
print("Reward:", reward[r])
print("\nEpsilon Greedy Strategy")
for i in range(5):
    x = random.random()
```

```
    if x < epsilon:
        a = random.choice(actions)
        print("Step", i + 1, ": Explore ->", a, "Reward =", reward[a])
    else:
        a = best
        print("Step", i + 1, ": Exploit ->", a, "Reward =", reward[a])
```

output:

```
========== RESTART: /Users/vinnu/Documents/e
Greedy Strategy
Selected Action: C
Reward: 10

Random Strategy
Selected Action: B
Reward: 8

Epsilon Greedy Strategy
Step 1 : Exploit -> C Reward = 10
Step 2 : Exploit -> C Reward = 10
Step 3 : Explore -> A Reward = 5
Step 4 : Explore -> A Reward = 5
Step 5 : Exploit -> C Reward = 10
```

Experiment 7: Multi-Armed Bandit Problem

program:

```
import random
reward = [4, 7, 10]
e = 0.2
print("Epsilon Greedy")
total1 = 0
for i in range(5):
    x = random.random()
    if x < e:
        arm = random.randint(0, 2)
    else:
        arm = 2
    print("Step", i + 1)
    print("Arm =", arm + 1)
    print("Reward =", reward[arm])
    total1 = total1 + reward[arm]
print("Total Reward =", total1)
print()
print("UCB")
total2 = 0
for i in range(5):
```

```
    if i == 0:
        arm = 0
    elif i == 1:
        arm = 1
    else:
        arm = 2
    print("Step", i + 1)
    print("Arm =", arm + 1)
    print("Reward =", reward[arm])
    total2 = total2 + reward[arm]
print("Total Reward =", total2)
```

Output:

```
========================================================== RESTART: /U
Epsilon Greedy
Step 1
Arm = 3
Reward = 10
Step 2
Arm = 3
Reward = 10
Step 3
Arm = 3
Reward = 10
Step 4
Arm = 3
Reward = 10
Step 5
Arm = 3
Reward = 10
Total Reward = 50

UCB
Step 1
Arm = 1
Reward = 4
Step 2
Arm = 2
Reward = 7
Step 3
Arm = 3
Reward = 10
Step 4
Arm = 3
Reward = 10
Step 5
Arm = 3
Reward = 10
Total Reward = 41
>
```

Experiment 8: Reward Visualization in Reinforcement Learning

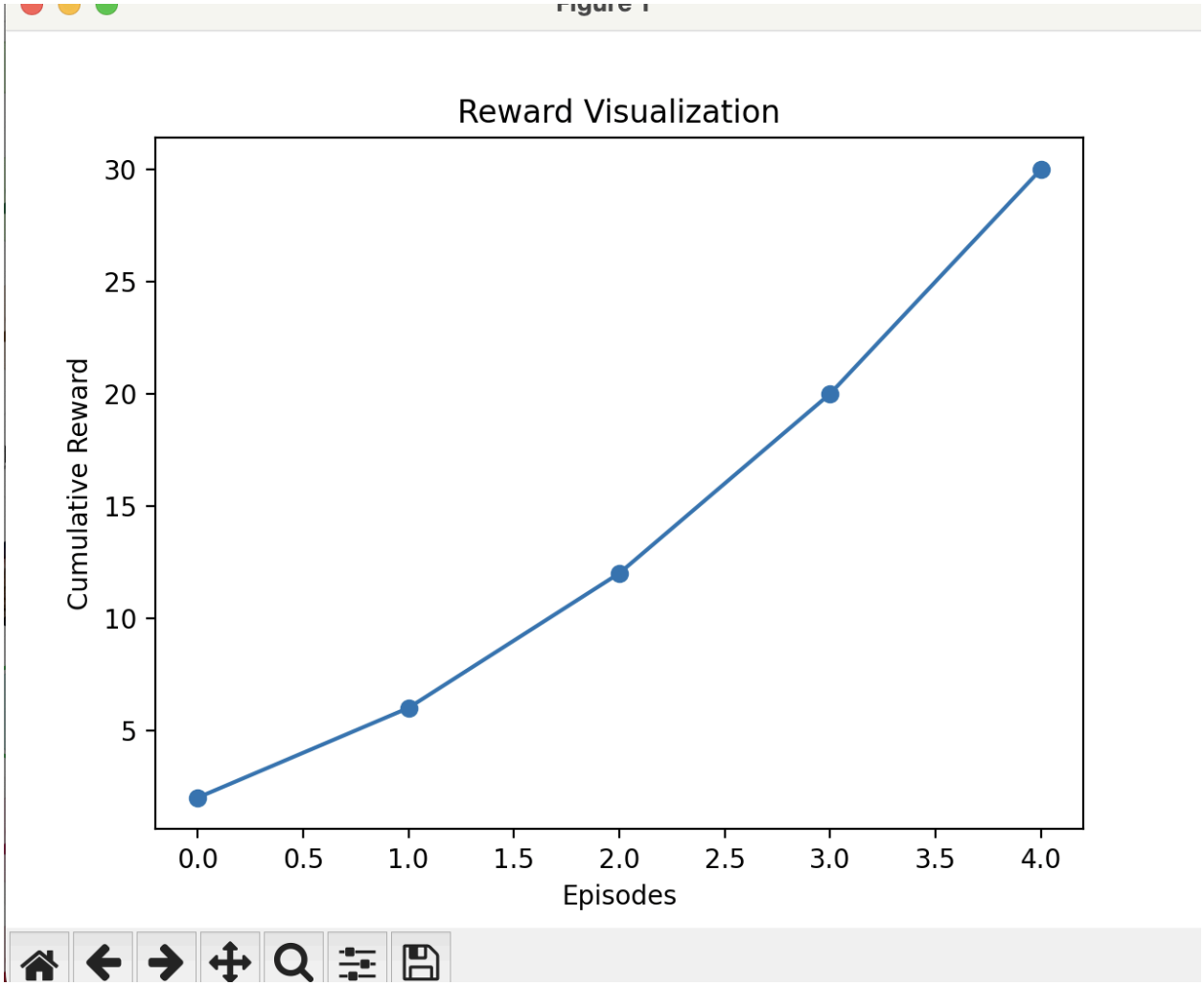
Program:

```
import matplotlib.pyplot as plt
rewards = [2, 4, 6, 8, 10]
cumulative = []
total = 0
for i in rewards:
    total = total + i
    cumulative.append(total)

print("Episode Rewards")
print(rewards)

print("Cumulative Rewards")
print(cumulative)
plt.plot(cumulative, marker="o")
plt.title("Reward Visualization")
plt.xlabel("Episodes")
plt.ylabel("Cumulative Reward")
plt.show()
```

Output:


Reward Visualization
Cumulative Reward
30
25
20
15
10
5
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Episodes

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Program:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
import numpy as np
x = np.array([
    [0.1, 0.2],
    [0.2, 0.3],
    [0.3, 0.4],
    [0.4, 0.5],
    [0.5, 0.6]
])
y = np.array([
    [0.3],
    [0.5],
    [0.7],
    [0.9],
    [1.1]
])
```

```
model = Sequential([
    Input(shape=(2,)),
    Dense(8, activation="relu"),
    Dense(1)
])
model.compile(optimizer="adam", loss="mse")

model.fit(x, y, epochs=100, verbose=0)
test = np.array([[0.35, 0.45]])
prediction = model.predict(test, verbose=0)
print("Predicted Value Function:")
print(prediction)
```

Output:

```
>
========== RESTART:
Predicted Value Function:
[[0.46864358]]
>
```

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Program:

```
import gymnasium as gym
import random

env = gym.make("CartPole-v1")

episodes = 5
success = 0

for i in range(episodes):
    state, info = env.reset()
    done = False
    total_reward = 0

    while not done:
        action = random.randint(0, 1)
        state, reward, terminated, truncated, info = env.step(action)

        total_reward += reward
```

```
        done = terminated or truncated

    print("Episode", i + 1)
    print("Reward:", total_reward)

    if total_reward >= 100:
        success += 1

print("\nSuccess Rate:", (success / episodes) * 100, "%")

env.close()
```

Output:

```
==================== RESTART: /Users/vinnu/
Episode 1
Reward: 25.0
Episode 2
Reward: 14.0
Episode 3
Reward: 16.0
Episode 4
Reward: 18.0
Episode 5
Reward: 9.0

Success Rate: 0.0 %
```